Visual-Inertial Odometry Estimator — Portfolio Summary

Three versions below, from shortest to longest, depending on where you're using it. All are written to be accurate to what's actually built and validated as of now — nothing here overstates results.

One-liner (LinkedIn headline / portfolio card)

> Built a monocular visual-inertial odometry (VIO) estimator in Python — error-state EKF fusing IMU pre-integration with a FAST/KLT visual front end — validated against synthetic ground-truth data, with a root-caused drift analysis driving an in-progress MSCKF-style upgrade.

Resume bullets

- Implemented a monocular visual-inertial odometry pipeline from scratch in Python: IMU pre-integration with bias Jacobians, an error-state Extended Kalman Filter, and a FAST/KLT/RANSAC visual front end
- Built a synthetic dataset generator with exact analytical ground truth (trajectory, IMU, and camera data) to validate the estimator end-to-end after encountering unreliable public-dataset hosting
- Diagnosed estimator drift through a structured series of isolated experiments (pure-inertial baseline, ground-truth-triangulation control, parallax sensitivity sweep), correctly identifying a correlated-bias feedback loop as the root cause rather than a implementation bug
- Currently implementing an MSCKF-style null-space measurement update (sliding-window pose augmentation, landmark marginalization via QR) to address the diagnosed limitation

---

Full project write-up (GitHub README / portfolio page)

Monocular Visual-Inertial Odometry Estimator

What it is: A from-scratch implementation of visual-inertial odometry (VIO) — the technique drones, AR headsets, and mobile robots use to track their own position and orientation by fusing an IMU (accelerometer + gyroscope) with a camera, without relying on GPS.

Core components (Python, NumPy/SciPy/OpenCV):

- **IMU pre-integration** with first-order bias Jacobians, letting the filter cheaply re-correct accumulated IMU measurements when its bias estimate updates, rather than re-integrating raw samples
- **Error-state Extended Kalman Filter (ESKF)** tracking a 15-dimensional state (position, velocity, orientation, gyro/accel bias) with orientation kept in its tangent space to avoid quaternion singularities
- **Visual front end**: FAST corner detection, pyramidal Lucas-Kanade (KLT) tracking, and RANSAC-based outlier rejection via essential-matrix estimation
- **Evaluation harness**: Absolute Trajectory Error (ATE) via Umeyama alignment and Relative Pose Error (RPE), implemented from scratch and matched against the standard `evo` toolkit's definitions

Validation approach: Public VIO benchmark datasets (EuRoC MAV, TUM VI) had unreliable hosting during development, so I built a synthetic data generator producing a flight trajectory with exact, analytically-known ground truth — position, orientation, simulated noisy IMU readings, and rendered camera frames with trackable features — all in the same file format the real-dataset loader expects. This let me validate every stage of the pipeline (propagation, pre-integration, the update step, the metrics themselves) against ground truth I controlled completely, via a 5-test unit suite plus full end-to-end pipeline runs.

Engineering process — diagnosing a real accuracy problem: Initial end-to-end runs showed several meters of drift over a 20-second trajectory. Rather than guess at fixes, I ran three targeted diagnostics: (1) confirmed pure-inertial dead reckoning diverges within seconds — expected physics, and proof the IMU mechanization was correct; (2) confirmed the EKF's landmark update math was correct by triangulating with *ground-truth* poses and showing it reliably reduced pose error; (3) swept a triangulation-baseline threshold and found no improvement, ruling out insufficient parallax as the cause. This isolated the actual root cause: the filter's landmark triangulation depends on its own (slightly uncertain) pose estimate, creating a correlated bias that doesn't average out across nearby frames the way independent sensor noise would.

Current work: Implementing an MSCKF (Multi-State Constraint Kalman Filter) update — a sliding window of cloned camera poses in the filter state, with feature observations processed via a null-space projection that algebraically removes the landmark's position from the update

entirely, so pose and landmark uncertainty are handled consistently instead of the landmark being treated as a known quantity.

Stack: Python, NumPy, SciPy, OpenCV. C++ port of the estimator core planned as a subsequent phase.

Notes for using any of the above

- If asked in an interview for a concrete accuracy number, the honest answer right now is "~3.4m ATE RMSE over 20s on synthetic ground-truth data, with a diagnosed root cause and a fix in progress" — not a EuRoC number yet. That's a completely fine thing to say and the diagnostic story is arguably more interesting than a bare metric.
- Update the "Current work" section once MSCKF is validated and swap in real accuracy numbers (before/after comparison) — that upgrade path is a strong narrative for a portfolio piece precisely because it shows iteration on a real, understood problem rather than a first-try result.